

基于RAG技术的智能知识库问答系统的设计与实现

学生姓名：[你的姓名]

学号：[你的学号]

专业：软件工程

指导教师：[导师姓名]

完成日期：2026年3月

摘要

在信息高度碎片化的时代，个人在学习、工作和生活中产生了大量分散于各个平台、各终端的文档、图片、视频及笔记等数据。这些宝贵的知识资产如果缺乏有效的管理与分析手段，不仅难以复用和共享，甚至容易造成重要信息的流失。个人知识管理（Personal Knowledge Management, PKM）已成为知识工程和人机交互领域的重要研究方向。尽管Notion、Evernote、Obsidian等工具在信息组织方面提供了较为成熟的解决方案，但在深度个性化、多模态数据统一管理以及智能化检索方面仍存在不足。

本文针对个人知识管理的实际需求，设计并实现了一个基于RAG（Retrieval-Augmented Generation，检索增强生成）技术的智能个人知识库问答系统。系统采用前后端分离架构，后端基于Spring Boot 3框架，集成Spring AI、MyBatis Plus、Spring Security等技术；前端采用Vue 3框架，结合Element Plus和Naive UI组件库。系统使用PostgreSQL的PgVector扩展作为向量数据库，MinIO作为对象存储，Redis作为缓存层，实现了多模态数据的统一存储与智能问答功能。

系统的核心功能包括：个人知识库管理、多格式文档向量化、RAG智能问答、内容管理、评论系统等。通过Apache Tika实现PDF、Word、TXT等多格式文档解析，使用TokenTextSplitter进行智能文本分割，结合OpenAI Embedding模型生成文档向量。在问答过程中，系统首先通过向量相似度检索相关知识片段，然后将检索结果作为上下文提供给大语言模型，生成准确且具有引用来源的个性化回答。

测试结果表明，系统能够准确理解用户问题，高效检索个人知识库中的相关内容，并生成高质量的回答。系统具有良好的可扩展性和用户体验，为个人知识沉淀、经历复盘和学习工作效率提升提供了有效的解决方案，帮助用户构建属于自己的"第二大脑"和网络世界中的个性化知识空间。

关键词： 个人知识管理；检索增强生成；向量数据库；智能问答；知识库管理；Spring AI；Vue 3

Abstract

In the era of highly fragmented information, individuals generate a large amount of documents, images, videos, and notes scattered across various platforms and terminals during study, work, and daily life. Without effective management and analysis tools, these valuable knowledge assets are difficult to reuse and share, and important information may even be lost. Personal Knowledge Management (PKM) has become an important research direction in the fields of knowledge engineering and human-computer interaction. Although tools such as Notion, Evernote, and Obsidian provide relatively mature solutions for information organization, they still have shortcomings in deep personalization, unified management of multimodal data, and intelligent retrieval.

This paper designs and implements an intelligent personal knowledge base Q&A system based on RAG (Retrieval-Augmented Generation) technology to address the actual needs of personal knowledge management. The system adopts a front-end and back-end separation architecture. The back-end is based on the Spring Boot 3 framework, integrating Spring AI, MyBatis Plus, Spring Security, and other technologies. The front-end uses the Vue 3 framework, combined with Element Plus and Naive UI component libraries. The system uses PostgreSQL's PgVector extension as the vector database, MinIO as object storage, and Redis as the caching layer, achieving unified storage of multimodal data and intelligent Q&A functions.

The core functions of the system include: personal knowledge base management, multi-format document vectorization, RAG intelligent Q&A, content management, and comment system. Multi-format document parsing (PDF, Word, TXT, etc.) is implemented through Apache Tika, intelligent text segmentation using TokenTextSplitter, and document vectors are generated using the OpenAI Embedding model. During the Q&A process, the system first retrieves relevant knowledge fragments through vector similarity search, then provides the retrieval results as context to the large language model to generate accurate and personalized answers with citation sources.

Test results show that the system can accurately understand user questions, efficiently retrieve relevant content from personal knowledge bases, and generate high-quality answers. The system has good scalability and user experience, providing an effective solution for personal knowledge沉淀, experience review, and improvement of learning and work efficiency, helping users build

their own "second brain" and personalized knowledge space in the network world.

Keywords: Personal Knowledge Management; Retrieval-Augmented Generation; Vector Database; Intelligent Q&A; Knowledge Base Management; Spring AI; Vue 3

目录

- **第一章 绪论**
 - 1.1 研究背景与意义
 - 1.2 国内外研究现状
 - 1.3 研究内容与目标
 - 1.4 论文组织结构
- **第二章 相关技术介绍**
 - 2.1 RAG技术原理
 - 2.2 向量数据库技术
 - 2.3 Spring Boot框架
 - 2.4 Vue 3框架
 - 2.5 其他关键技术
- **第三章 系统需求分析**
 - 3.1 功能需求分析
 - 3.2 非功能需求分析
 - 3.3 用例分析
- **第四章 系统设计**
 - 4.1 系统架构设计

- 4.2 数据库设计
 - 4.3 核心模块设计
 - 4.4 UML类图设计
- 第五章 系统实现
 - 5.1 RAG问答模块实现
 - 5.2 知识库管理模块实现
 - 5.3 文档向量化实现
 - 5.4 前端界面实现
- 第六章 系统测试
 - 6.1 测试环境
 - 6.2 功能测试
 - 6.3 性能测试
- 第七章 总结与展望
 - 7.1 工作总结
 - 7.2 未来展望
- 参考文献
- 致谢

第一章 绪论

1.1 研究背景与意义

1.1.1 研究背景

在信息高度碎片化的时代，个人在学习、工作和生活中产生了大量分散于各个平台、各终端的文档、图片、视频及笔记等数据。这些宝贵的个人知识资产如果缺乏有效的管理与分析手段，不仅难以复用和共享，甚至容易造成重要信息的流失。正如知识管理专家所言，“人在学

习过程中最可怕的是遗忘和生疏"，如果没有一种维持自己知识状态的有效机制，所学习过的知识、难忘的经历终究会在时间的磨砺下消耗殆尽。

个人知识管理（Personal Knowledge Management, PKM）旨在帮助个体有效地获取、组织、存储和检索知识。然而，现有的个人知识管理工具和方法仍存在以下问题：

1. **知识碎片化严重**：个人资料分散在社交媒体、云盘、笔记软件、本地存储等多个平台，缺乏统一的管理入口，难以形成系统化的知识网络。
2. **检索方式落后**：传统工具主要依赖关键词搜索和文件夹分类，无法理解用户问题的真实意图，导致检索效率低下。当用户想要回顾某段经历或查找特定知识点时，往往需要耗费大量时间翻阅。
3. **缺乏智能交互**：现有系统只能返回存储的原始内容，无法基于已有知识进行智能问答和深度分析，用户需要自行消化和理解大量信息。
4. **多模态支持不足**：大多数工具侧重于文本管理，对图片、视频等多模态内容的支持有限，难以全面记录和回顾个人经历。

近年来，大语言模型（Large Language Model, LLM）如GPT-4、Claude等的出现，为个人知识管理带来了新的可能。通过AI技术，用户可以用自然语言与知识库进行对话，快速获取所需信息。然而，纯粹依赖大语言模型也存在局限性：模型缺乏用户个人的专属知识，无法基于用户的实际经历和积累提供个性化回答。

检索增强生成（RAG）技术的出现为上述问题提供了有效解决方案。RAG通过将个人知识库的向量检索与大语言模型相结合，既保留了LLM强大的语言理解和生成能力，又能够基于用户自己的知识资产提供个性化、可追溯的回答，帮助用户构建属于自己的"第二大脑"。

1.1.2 研究意义

本研究的理论意义在于：

1. **探索RAG技术在个人知识管理中的应用**：研究如何将RAG技术与个人知识库系统结合，实现基于个人知识资产的智能问答，填补通用大模型缺乏个人专属知识的空白。
2. **个人知识图谱的构建方法**：探索如何整合个人分散的多模态数据（文本、图片、视频等），构建结构化的个人知识网络。
3. **向量数据库在个人级数据管理中的应用**：探索PgVector等向量数据库在个人知识库场景下的应用方法和性能优化策略。

本研究的实际意义在于：

1. **构建个人"第二大脑"**：帮助用户将分散在各平台的知识资产统一管理，形成可随时访问、检索的个人知识库，有效对抗知识遗忘。
2. **实现自我认知与复盘**：通过AI智能问答，用户可以基于自己的经历和学习记录进行深度反思和总结，促进个人成长。
3. **打造个性化网络身份**：用户可以通过经营自己的知识平台，展示专业领域和人生经历，让外界快速了解自己，构建网络世界的个人品牌。
4. **提升学习工作效率**：通过自然语言交互快速检索个人知识，无需记忆文件位置或翻阅大量资料，显著提升信息利用效率。

1.2 国内外研究现状

1.2.1 智能问答系统与个人知识管理研究现状

(1) 智能问答系统发展

智能问答系统的研究可以追溯到20世纪60年代的ELIZA系统。随着技术的发展，问答系统经历了以下几个阶段：

1. **基于规则的问答系统：**早期系统主要依赖人工编写的规则和模板，如ELIZA、ALICE等。这类系统灵活性差，难以处理复杂问题。
2. **基于信息检索的问答系统：**利用搜索引擎技术检索相关文档，然后从中提取答案。代表性系统包括IBM的Watson、微软的Bing等。
3. **基于深度学习的问答系统：**利用神经网络模型进行语义理解和答案生成。如BERT、GPT等预训练模型的出现大幅提升了问答系统的性能。
4. **基于RAG的问答系统：**结合检索和生成技术，既能利用外部知识库，又能生成流畅的自然语言回答。如Facebook的RAG模型、OpenAI的GPT-4 with Retrieval等。

(2) 个人知识管理（PKM）工具发展

国外在个人知识管理工具方面起步较早，已形成较为成熟的产品生态：

- **Notion：**提供灵活的块状编辑和数据库功能，支持多平台同步，但在智能检索和语义理解方面能力有限。
- **Evernote：**老牌笔记软件，支持多格式内容存储和标签管理，但缺乏AI辅助分析功能。
- **Obsidian：**基于本地Markdown的双向链接笔记工具，适合构建知识网络，但对多模态内容支持不足。
- **Mem.ai：**引入AI辅助的笔记工具，支持自动标签和智能搜索，但在深度个性化方面仍有局限。

国内方面，语雀、石墨文档、飞书文档等协作工具也逐步加入个人知识管理功能，但多数仍侧重于团队协作而非个人知识资产的深度管理。现有工具普遍存在智能化程度不足、多模态支持有限、难以基于个人知识进行深度交互等问题。

1.2.2 RAG技术研究现状

RAG技术最早由Facebook AI Research在2020年提出。其核心思想是将检索模块与生成模块结合：

1. **检索模块**：使用Dense Passage Retrieval (DPR) 等技术，将文档和问题编码为向量，通过向量相似度检索相关文档。
2. **生成模块**：将检索到的文档作为上下文，输入到生成模型（如BART、T5）中，生成最终答案。

近年来，RAG技术得到了快速发展：

- **LangChain**：提供了完整的RAG开发框架，支持多种向量数据库和LLM。
- **LlamaIndex**：专注于文档索引和检索，提供了丰富的数据连接器。
- **Spring AI**：Spring生态系统中的AI框架，提供了RAG的Java实现。

国内方面，百度的文心一言、阿里的通义千问等也都集成了RAG功能，用于增强模型的知识获取能力。

1.2.3 向量数据库研究现状

向量数据库是RAG系统的核心组件，用于存储和检索高维向量。主流的向量数据库包括：

1. 专用向量数据库：

- Pinecone：云原生向量数据库，提供高性能的相似度搜索。
- Milvus：开源向量数据库，支持多种索引算法。
- Weaviate：支持语义搜索和混合搜索的向量数据库。

2. 传统数据库的向量扩展：

- PgVector：PostgreSQL的向量扩展，支持向量存储和相似度搜索。
- Elasticsearch：通过dense_vector类型支持向量检索。
- Redis：通过RediSearch模块支持向量搜索。

本系统选择PgVector作为向量数据库，主要考虑到其与PostgreSQL的无缝集成，可以在同一数据库中管理结构化数据和向量数据，简化系统架构。

1.3 研究内容与目标

本研究围绕基于RAG技术的个人知识库问答系统展开，主要研究内容包括RAG技术的应用研究、系统架构设计、关键功能实现以及系统测试与优化四个方面。在RAG技术应用方面，将深入研究RAG的工作原理和实现方法，重点探索文档分割、向量化、检索等关键技术，并优化检索策略以提高答案的准确性。在系统架构设计方面，采用前后端分离的架构模式，设计知识库管理、文档处理、问答交互等核心模块，同时完成数据库模型和API接口的设计。在关键功能实现方面，将实现多格式文档的解析和向量化处理、基于向量相似度的文档检索、流式RAG问答以及知识库管理、内容管理等辅助功能。在系统测试与优化方面，将进行全面的测试以验证系统的正确性，开展性能测试以优化系统响应速度，并收集用户反馈以持续改进用户体验。

本研究的目标是设计并实现一个面向个人用户的、功能完善的智能知识库问答系统。具体而言，系统应支持文本、图片、视频等多种格式个人数据的统一存储、管理和向量化处理，实现多模态知识的有效整合；能够基于用户的个人知识库准确理解自然语言问题，生成个性化、可追溯的智能化回答；能够帮助用户基于自身经历和知识进行复盘与总结，支持个人成长和自我认知；能够实时处理用户请求，将响应时间控

制在3秒以内，提供流畅的交互体验；同时提供友好的用户界面，支持对话式交互，使用户无需复杂操作即可便捷地管理和检索个人知识。

1.4 应解决的主要问题及应达到的要求

本研究需要解决的主要问题包括：首先，如何有效整合个人分散在多平台的多模态数据（文本、图片、视频等），实现统一的知识表示与存储，解决知识碎片化严重的问题；其次，如何利用向量嵌入和相似度检索技术，实现基于语义理解的自然语言检索，替代传统关键词搜索；再次，如何将RAG技术与个人知识库结合，基于用户自身积累的知识资产提供个性化、可追溯的智能问答；最后，如何设计友好的用户界面和交互方式，降低用户使用门槛，实现真正的"第二大脑"体验。

本研究应达到的要求包括：在功能层面，系统应支持多格式文档的自动解析与向量化、基于向量相似度的高效检索、流式RAG问答以及内容管理等功能；在性能层面，系统响应时间应控制在3秒以内，支持不少于100个并发用户访问，向量检索时间不超过500毫秒；在用户体验层面，系统应提供对话式交互界面，支持自然语言提问，无需复杂操作即可完成知识管理和检索；在可靠性层面，系统应保证99%以上的可用性，具备完善的异常处理和日志记录机制。

第二章 相关技术介绍

2.1 RAG技术原理

2.1.1 RAG技术概述

检索增强生成（Retrieval-Augmented Generation, RAG）是一种结合信息检索和文本生成的技术。其核心思想是在生成回答之前，先从外部知识库中检索相关信息，然后将检索到的信息作为上下文提供给生成模型，从而提高回答的准确性和可靠性。

RAG技术的工作流程如下：

1. **问题编码**：将用户的问题转换为向量表示。
2. **文档检索**：在向量数据库中检索与问题向量相似的文档片段。
3. **上下文构建**：将检索到的文档片段组织成上下文。
4. **答案生成**：将问题和上下文一起输入到大语言模型，生成最终答案。

2.1.2 向量嵌入技术

向量嵌入（Embedding）是将文本转换为高维向量的技术。在RAG系统中，文档和问题都需要转换为向量，以便进行相似度计算。

常用的嵌入模型包括：

- **OpenAI Embedding**：OpenAI提供的text-embedding-ada-002模型，生成1536维向量，具有良好的语义表示能力。
- **BERT系列**：如sentence-transformers，支持多语言，可以本地部署。
- **BGE系列**：智源研究院开源的中文嵌入模型，在中文任务上表现优异。

本系统使用OpenAI Embedding模型，主要考虑其高质量的语义表示和与OpenAI LLM的良好兼容性。

2.1.3 相似度检索算法

向量相似度检索是RAG的核心步骤。常用的相似度度量方法包括：

1. **余弦相似度（Cosine Similarity）**：

$$1 \text{ similarity} = (A \cdot B) / (||A|| \times ||B||)$$

取值范围[-1, 1]，值越大表示越相似。

2. **欧氏距离（Euclidean Distance）**：

$$1 \text{ distance} = \sqrt{(\sum(A_i - B_i)^2)}$$

距离越小表示越相似。

3. 内积 (Inner Product) :

$$1 \text{ similarity} = A \cdot B$$

适用于归一化向量。

本系统默认使用余弦相似度，因为它对向量的长度不敏感，更适合语义相似度计算。

2.1.4 文档分割策略

由于大语言模型的上下文窗口有限，需要将长文档分割成较小的片段 (Chunk) 。常用的分割策略包括：

1. **固定长度分割**：按字符数或token数分割，简单但可能破坏语义完整性。
2. **句子分割**：按句子边界分割，保持语义完整性。
3. **段落分割**：按段落分割，适合结构化文档。
4. **递归分割**：先按段落分割，如果片段过大再按句子分割，兼顾语义和长度。

本系统使用TokenTextSplitter，按token数量分割，同时设置重叠 (overlap) 以保持上下文连贯性。

2.2 向量数据库技术

2.2.1 PgVector简介

PgVector是PostgreSQL的一个扩展，为PostgreSQL添加了向量数据类型和向量操作功能。其主要特点包括：

1. **无缝集成**：作为PostgreSQL扩展，可以在同一数据库中管理结构化数据和向量数据。
2. **标准SQL接口**：使用标准SQL语法进行向量操作，学习成本低。
3. **多种索引算法**：支持IVFFlat、HNSW等索引算法，提高检索效率。
4. **ACID保证**：继承PostgreSQL的事务特性，保证数据一致性。

2.2.2 向量索引算法

为了提高大规模向量检索的效率，PgVector支持多种索引算法：

1. IVFFlat (Inverted File with Flat compression) :

- 将向量空间划分为多个聚类
- 检索时只搜索最近的几个聚类
- 适合中等规模数据集（百万级）

2. HNSW (Hierarchical Navigable Small World) :

- 构建多层图结构
- 检索时从顶层开始逐层搜索
- 检索速度快，适合大规模数据集（千万级）

本系统在数据量较小时使用暴力搜索，数据量增大后可创建HNSW索引以提高性能。

2.3 Spring Boot框架

Spring Boot是基于Spring框架的快速开发框架，提供了自动配置、起步依赖、内嵌服务器等特性，大幅简化了Spring应用的开发。本系统使用Spring Boot 3.3.5版本，基于Java 21，主要利用了自动配置特性来自动配置数据源、JPA、Security等组件，通过起步依赖简化依

赖管理，使用内嵌Tomcat服务器无需单独部署，同时利用Actuator端点监控应用状态。

Spring AI是Spring生态系统中的AI框架，提供了与各种AI服务集成的能力，其主要组件包括ChatModel统一聊天模型接口、EmbeddingModel嵌入模型接口、VectorStore向量存储接口、DocumentReader文档读取器以及Advisor顾问模式。ChatModel支持OpenAI、Azure OpenAI、Anthropic等多种模型服务，EmbeddingModel用于生成文本向量，VectorStore支持PgVector、Pinecone、Milvus等向量数据库，DocumentReader支持PDF、Word、TXT等多种文档格式解析，Advisor则用于增强对话能力，如RAG、记忆管理等。本系统使用Spring AI 1.0.0-M4版本，实现了RAG问答的核心功能。

MyBatis Plus是MyBatis的增强工具，提供了CRUD接口、分页插件、代码生成器等功能，简化了数据库操作。其主要特性包括BaseMapper提供基础CRUD方法无需编写SQL，条件构造器提供链式API构建查询条件，分页插件自动处理分页查询，以及逻辑删除功能支持不物理删除数据。

2.4 Vue 3框架与前端技术栈

Vue 3是流行的前端框架，采用组合式API（Composition API），提供了更好的TypeScript支持和性能优化。其主要特性包括基于Proxy的响应式系统，相比Vue 2的Object.defineProperty具有更好的性能表现；组合式API使用setup函数组织逻辑，使代码结构更清晰；Teleport特性允许将组件渲染到DOM的其他位置，便于实现模态框等功能；Suspense则用于处理异步组件的加载状态，提升用户体验。

Vite是新一代前端构建工具，基于ES模块提供了极快的开发服务器启动速度和热更新能力。其优势在于利用浏览器原生ES模块实现快速冷启动无需打包，修改代码后即时热更新立即生效，同时使用Rollup进行生产构建输出优化后的代码。本系统使用Vite作为构建工具，显著提升

了开发效率。

在UI组件库方面，本系统使用了Element Plus和Naive UI两个组件库。Element Plus是基于Vue 3的组件库，提供了丰富的表单、表格、对话框等组件，适合后台管理系统开发；Naive UI是轻量级Vue 3组件库，提供了美观的UI设计和良好的TypeScript支持。此外，本系统还使用了GSAP（GreenSock Animation Platform）动画库来实现流畅的页面过渡效果和交互动画，包括页面切换动画、滚动触发动画以及元素状态过渡效果，显著提升了用户界面的视觉体验和交互质感。

2.5 其他关键技术

MinIO是开源的对象存储服务，兼容Amazon S3 API，本系统使用MinIO存储文档、图片、视频等文件。其主要特性包括支持高并发读写的高性能架构，支持分布式部署实现高可用，以及兼容AWS S3 API便于与其他系统集成和迁移。

Redis是高性能的内存数据库，本系统使用Redis作为缓存层来缓存热点数据，减少数据库查询压力。主要应用场景包括内容列表缓存如文章列表和推荐内容，统计数据缓存如访问统计和活动统计，以及标签缓存如内容标签和分类信息。

Apache Tika是文档解析工具，支持PDF、Word、Excel、PPT等多种格式的文档处理。本系统使用Tika实现文档内容提取以获取文本内容，元数据提取以获取标题、作者等信息，以及自动检测文档格式以确保正确处理不同类型的文件。

第三章 系统需求分析

3.1 功能需求分析

3.1.1 用户角色分析

本系统主要面向两类用户：

1. 普通用户：

- 浏览内容（文章、游戏、学习资源、视频）
- 使用AI问答功能
- 发表评论
- 查看个人信息

2. 管理员：

- 管理内容（创建、编辑、删除）
- 管理知识库（创建、上传文档、删除）
- 管理评论（审核、推荐、删除）
- 查看统计数据
- 管理分类和标签

3.1.2 核心功能需求

1. AI智能问答功能

- **简单对话：**用户可以与AI进行简单的对话交互，无需知识库支持。
- **RAG问答：**基于知识库的智能问答，系统检索相关文档后生成回答。
- **引用来源：**显示回答的引用来源，用户可以查看原始文档。
- **多模态对话：**支持上传图片进行问答。
- **对话历史：**保存对话记录，支持查看历史对话。
- **流式输出：**实时显示AI生成的回答，提升用户体验。

2. 知识库管理功能

- **知识库创建**：管理员可以创建多个知识库，每个知识库有独立的名称和描述。
- **文档上传**：支持上传PDF、Word、TXT等多种格式的文档。
- **文档解析**：自动解析文档内容，提取文本和图片。
- **向量化处理**：自动将文档内容转换为向量，存储到向量数据库。
- **文档管理**：查看、删除知识库中的文档。
- **知识库切换**：用户可以选择不同的知识库进行问答。

3. 内容管理功能

- **内容创建**：管理员可以创建文章、游戏、学习资源、视频等内容。
- **富文本编辑**：支持Markdown格式编辑内容。
- **封面上传**：支持上传内容封面图片。
- **视频管理**：支持上传视频文件或添加视频链接。
- **分类管理**：内容按分类组织（文章、游戏、学习、视频）。
- **标签管理**：为内容添加标签，方便检索。
- **推荐设置**：设置推荐内容，在首页展示。
- **内容同步**：内容自动同步到知识库，支持AI问答。

4. 评论系统功能

- **发表评论**：用户可以对内容发表评论。
- **头像上传**：支持上传评论者头像。
- **评论审核**：管理员可以审核评论。

- **评论推荐：**管理员可以设置推荐评论。
- **评论同步：**评论自动同步到知识库，丰富知识内容。

5. 数据统计功能

- **访问统计：**统计内容的浏览量。
- **活动热力图：**显示用户活动的时间分布。
- **AI助手报告：**AI自动生成数据分析报告。
- **快捷信息：**显示系统关键指标。

3.1.3 辅助功能需求

1. 用户认证与授权

- **用户注册：**用户可以注册账号。
- **用户登录：**支持用户名密码登录。
- **JWT认证：**使用JWT实现无状态认证。
- **角色权限：**基于角色的访问控制（RBAC）。

2. 文件存储管理

- **对象存储：**使用MinIO存储文件。
- **多桶管理：**不同类型文件存储在不同的桶中。
- **临时访问：**生成临时访问URL，有效期7天。
- **文件校验：**使用MD5校验文件完整性。

3. 缓存管理

- **热点数据缓存：**缓存常用数据，减少数据库查询。
- **缓存过期：**设置合理的缓存过期时间。

- **缓存更新：**数据变更时自动清除相关缓存。

3.2 非功能需求分析

3.2.1 性能需求

1. 响应时间：

- 普通页面加载时间 < 1秒
- AI问答响应时间 < 3秒（首字响应 < 1秒）
- 文档上传处理时间 < 10秒（取决于文档大小）

2. 并发能力：

- 支持100个并发用户同时访问
- 支持10个并发AI问答请求

3. 数据库性能：

- 向量检索时间 < 500ms
- 普通查询时间 < 100ms

3.2.2 可靠性需求

1. **系统可用性：**系统可用性达到99%以上。
2. **数据备份：**定期备份数据库和文件存储。
3. **错误处理：**完善的异常处理机制，避免系统崩溃。
4. **日志记录：**记录系统运行日志，便于问题排查。

3.2.3 安全性需求

1. **身份认证：**使用JWT进行身份认证。
2. **密码加密：**使用BCrypt加密用户密码。
3. **权限控制：**基于角色的访问控制。
4. **数据加密：**敏感数据加密存储。

5. **防护措施**：防止SQL注入、XSS攻击等。

3.2.4 可扩展性需求

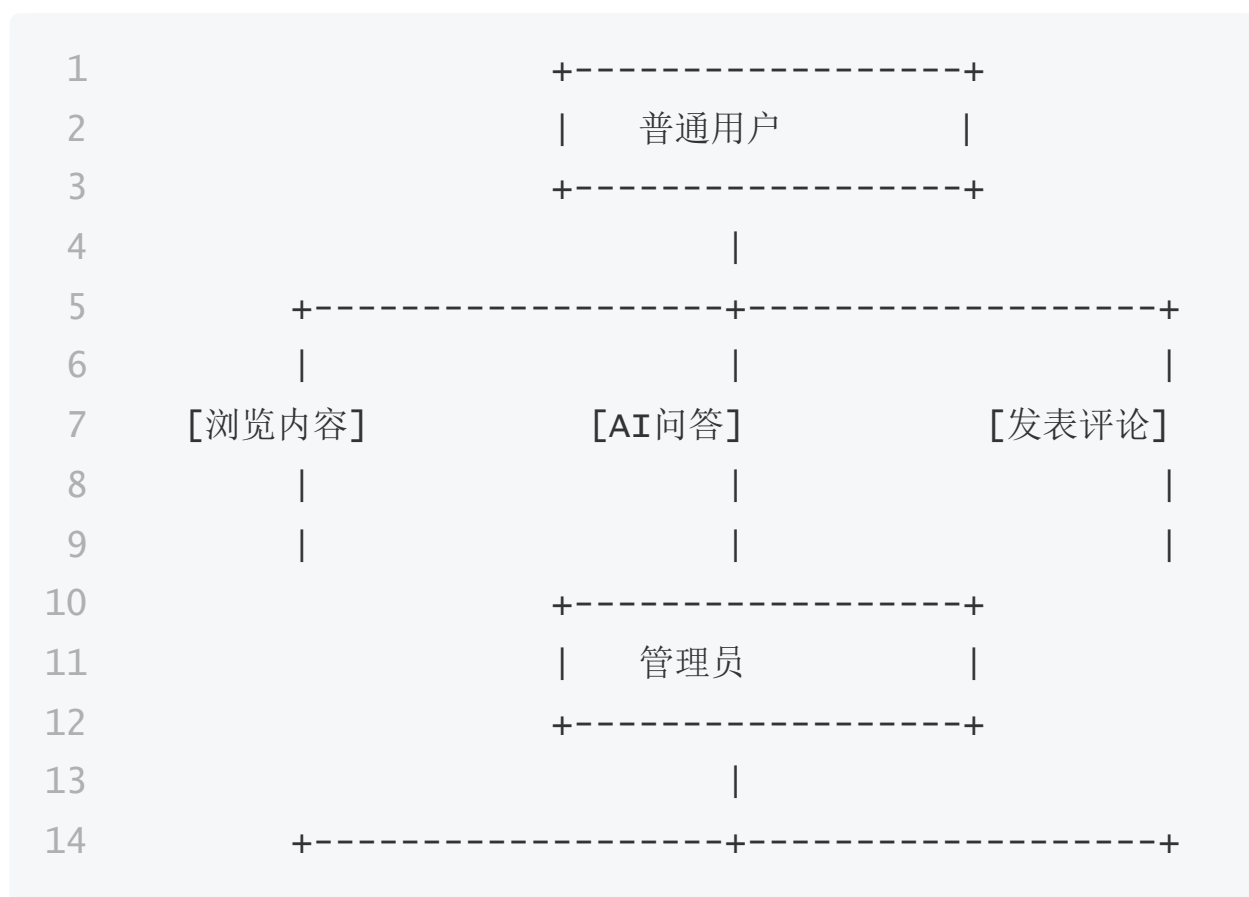
1. **模块化设计**：系统采用模块化设计，便于功能扩展。
2. **多知识库支持**：支持创建多个知识库。
3. **多模型支持**：支持配置不同的LLM模型。
4. **插件化**：支持添加新的文档解析器、向量数据库等。

3.2.5 易用性需求

1. **友好界面**：提供简洁美观的用户界面。
2. **响应式设计**：支持PC和移动端访问。
3. **操作提示**：提供清晰的操作提示和错误信息。
4. **帮助文档**：提供系统使用文档。

3.3 用例分析

3.3.1 用例图



15			
16	[管理内容]	[管理知识库]	[管理评论]
17			
18	[查看统计]	[上传文档]	[审核评论]

3.3.2 核心用例描述

用例1：AI智能问答

- **用例名称：**AI智能问答
- **参与者：**普通用户
- **前置条件：**用户已登录系统
- **主要流程：**
 - 1. 用户在问答页面输入问题
 - 2. 用户选择知识库（可选）
 - 3. 用户点击发送按钮
 - 4. 系统将问题转换为向量
 - 5. 系统在向量数据库中检索相关文档
 - 6. 系统将问题和检索结果发送给LLM
 - 7. 系统流式返回AI生成的回答
 - 8. 系统显示引用来源
- **后置条件：**对话记录保存到数据库
- **异常流程：**
 - 如果检索不到相关文档，系统提示"未找到相关信息"
 - 如果LLM调用失败，系统提示"服务暂时不可用"

用例2：上传文档到知识库

- **用例名称：**上传文档到知识库
- **参与者：**管理员
- **前置条件：**管理员已登录系统，知识库已创建
- **主要流程：**
 1. 管理员进入知识库管理页面
 2. 管理员选择目标知识库
 3. 管理员点击上传按钮，选择文档文件
 4. 系统上传文件到MinIO
 5. 系统使用Tika解析文档内容
 6. 系统使用TokenTextSplitter分割文档
 7. 系统使用Embedding模型生成向量
 8. 系统将向量存储到PgVector
 9. 系统更新文档状态为"已向量化"
- **后置条件：**文档可用于AI问答
- **异常流程：**
 - 如果文件格式不支持，系统提示"不支持的文件格式"
 - 如果向量化失败，系统提示"文档处理失败"

用例3：创建内容

- **用例名称：**创建内容
- **参与者：**管理员
- **前置条件：**管理员已登录系统
- **主要流程：**
 1. 管理员进入内容管理页面

2. 管理员点击创建按钮
 3. 管理员填写标题、描述、内容等信息
 4. 管理员选择分类和标签
 5. 管理员上传封面图片（可选）
 6. 管理员点击保存按钮
 7. 系统保存内容到数据库
 8. 系统将内容同步到知识库
 9. 系统清除相关缓存
- **后置条件：**内容可被用户浏览，可用于AI问答
 - **异常流程：**
 - 如果必填字段为空，系统提示"请填写完整信息"
 - 如果同步知识库失败，系统记录日志但不影响内容创建

用例4：发表评论

- **用例名称：**发表评论
- **参与者：**普通用户
- **前置条件：**用户已登录系统，内容已存在
- **主要流程：**
 1. 用户浏览内容详情页
 2. 用户在评论区输入评论内容
 3. 用户上传头像（可选）
 4. 用户点击提交按钮
 5. 系统保存评论到数据库
 6. 系统更新内容的评论数

7. 系统将评论同步到知识库

8. 系统清除相关缓存

- **后置条件：**评论显示在内容详情页，可用于AI问答
- **异常流程：**
 - 如果评论内容为空，系统提示"请输入评论内容"
 - 如果头像上传失败，使用默认头像

3.3.3 系统边界

本系统的边界包括：

系统内部：

- AI问答功能
- 知识库管理
- 内容管理
- 评论系统
- 用户认证与授权
- 数据统计

系统外部：

- OpenAI API（LLM和Embedding服务）
 - MinIO对象存储
 - PostgreSQL数据库
 - Redis缓存
 - MySQL数据库
-

第四章 系统设计（续）

4.2 数据库设计

本系统采用PostgreSQL作为主数据库，配合PgVector扩展实现向量数据存储。数据库设计遵循第三范式，同时考虑查询性能优化，建立了合理的索引和触发器机制。

4.2.1 数据库架构设计

系统数据库采用多表结构，主要包括以下核心数据表：

1. 用户权限相关表

- **system_user**: 系统用户表，存储用户基本信息（用户名、密码等）
- **system_role**: 系统角色表，定义用户角色（管理员、普通用户）
- **system_permission**: 系统权限表，定义具体操作权限
- **system_user_role**: 用户-角色关联表，实现多对多关系
- **system_role_permission**: 角色-权限关联表，实现RBAC权限控制

2. 知识库相关表

- **knowledge_base**: 知识库表，存储知识库名称、描述等元数据
- **document_entity**: 文档实体表，存储文档文件信息、向量化状态
- **origin_file_source**: 原始文件资源表，存储MinIO文件元数据（文件名、存储桶、对象名、MD5等）

3. 内容管理相关表

- **content**: 内容表，存储文章、游戏、学习资源、视频等（标题、分类、封面、浏览量、评论数等）
- **comment**: 评论表，存储用户评论（内容ID、评论者、评论内容、父评论ID支持回复嵌套）
- **murmur**: 碎碎念表，存储用户动态短内容
- **photo**: 照片表，存储照片元数据（存储桶名称、对象名称、缩略图名称）

4. AI对话相关表

- **chat_conversation**: 对话会话表，存储会话标题、用户ID、知识库ID
- **chat_message**: 聊天消息表，存储消息内容、角色（user/assistant）、附件信息、消息序号

4.2.2 数据库表结构设计

1. 用户权限相关表

表4.1 system_user表（系统用户表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
username	VARCHAR(255)	用户名
password	VARCHAR(255)	密码（BCrypt加密）
create_time	TIMESTAMP	创建时间

字段名	类型	说明
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.2 system_role表（系统角色表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
name	VARCHAR(255)	角色名
description	VARCHAR(500)	角色描述
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.3 system_permission表（系统权限表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
name	VARCHAR(255)	权限名称
description	VARCHAR(500)	权限描述
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.4 system_user_role表（用户-角色关联表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
user_id	BIGINT	用户ID（外键）
role_id	BIGINT	角色ID（外键）
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志

字段名	类型	说明
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.5 system_role_permission表（角色-权限关联表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
role_id	BIGINT	角色ID（外键）
permission_id	BIGINT	权限ID（外键）
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

2. 知识库相关表

表4.6 knowledge_base表（知识库表）

字段名	类型	说明
id	VARCHAR(32)	主键，UUID

字段名	类型	说明
name	VARCHAR(100)	知识库名称
description	TEXT	知识库描述
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.7 document_entity表（文档实体表）

字段名	类型	说明
id	BIGSERIAL	主键，自增
file_name	VARCHAR(512)	文件名
path	TEXT	文件路径
base_id	VARCHAR(32)	所属知识库ID（外键）
is_embedding	BOOLEAN	是否已向量化
resource_id	VARCHAR(64)	关联文件资源ID
create_time	TIMESTAMP	创建时间

字段名	类型	说明
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.8 origin_file_source表（原始文件资源表）

字段名	类型	说明
id	TEXT	主键，文件唯一标识
file_name	TEXT	文件名
path	TEXT	文件存储路径
is_image	BOOLEAN	是否为图片文件
bucket_name	TEXT	MinIO存储桶名称
object_name	TEXT	MinIO对象名称
content_type	TEXT	文件MIME类型
size	BIGINT	文件大小（字节）
md5	TEXT	文件MD5哈希值
images	TEXT	文档内图片列表 (JSON)

字段名	类型	说明
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

3. 内容管理相关表

表4.9 content表（内容表）

字段名	类型	说明
id	VARCHAR(32)	主键，UUID
title	VARCHAR(255)	标题
description	TEXT	简介/描述
category	VARCHAR(50)	分类 (article/game/study/video)
cover	TEXT	封面图片URL
content	TEXT	正文内容（Markdown）
comment_count	INT	评论数
view_count	INT	浏览量

字段名	类型	说明
is_recommend	BOOLEAN	是否推荐
content_type	VARCHAR(20)	内容类型
video_type	VARCHAR(10)	视频类型
video_url	VARCHAR(500)	视频URL
video_duration	INT	视频时长（秒）
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.10 comment表（评论表）

字段名	类型	说明
id	VARCHAR(32)	主键，UUID
content_id	VARCHAR(32)	关联内容ID
content_type	VARCHAR(50)	内容类型
avatar	TEXT	评论者头像URL
author	VARCHAR(255)	评论者昵称

字段名	类型	说明
user_id	VARCHAR(255)	评论者用户ID
content	TEXT	评论内容
parent_id	VARCHAR(32)	父评论ID（回复用）
is_recommend	BOOLEAN	是否推荐
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.11 murmur表（碎碎念表）

字段名	类型	说明
id	VARCHAR(32)	主键，UUID
text	TEXT	碎碎念内容
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志

字段名	类型	说明
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.12 photo表（照片表）

字段名	类型	说明
id	VARCHAR(32)	主键，UUID
bucket_name	VARCHAR(100)	MinIO存储桶名称
object_name	TEXT	MinIO对象名称（原图）
thumbnail_name	VARCHAR(500)	缩略图对象名称
description	TEXT	照片描述
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

4. AI对话相关表

表4.13 chat_conversation表（对话会话表）

字段名	类型	说明
id	TEXT	主键，UUID
title	TEXT	会话标题
user_id	BIGINT	发起人用户ID (外键)
knowledge_base_id	VARCHAR(255)	关联知识库ID
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

表4.14 chat_message表（聊天消息表）

字段名	类型	说明
id	TEXT	主键，UUID
conversation_id	TEXT	所属会话ID
message_no	INT	消息序号
content	TEXT	消息内容

字段名	类型	说明
role	TEXT	角色 (user/assistant)
has_media	BOOLEAN	是否携带附件
resource_ids	TEXT	附件资源ID列表 (JSON)
is_clean	BOOLEAN	是否已清理
create_time	TIMESTAMP	创建时间
update_time	TIMESTAMP	更新时间
deleted	BOOLEAN	逻辑删除标志
creator	VARCHAR(255)	创建人
updater	VARCHAR(255)	更新人

4.2.3 数据库优化设计

1. 索引设计

为提高查询效率，系统为常用查询字段建立了索引：

```
1 CREATE INDEX "idx_chat_message_conversation_id" ON
  "chat_message"(conversation_id);
2 CREATE INDEX "idx_chat_conversation_user_id" ON
  "chat_conversation"(user_id);
3 CREATE INDEX "idx_document_entity_base_id" ON
  "document_entity"(base_id);
4 CREATE INDEX "idx_content_category" ON "content"
  (category);
5 CREATE INDEX "idx_comment_content" ON "comment"
  (content_id, content_type);
```

2. 自动更新时间触发器

系统使用PostgreSQL触发器实现update_time字段的自动更新：

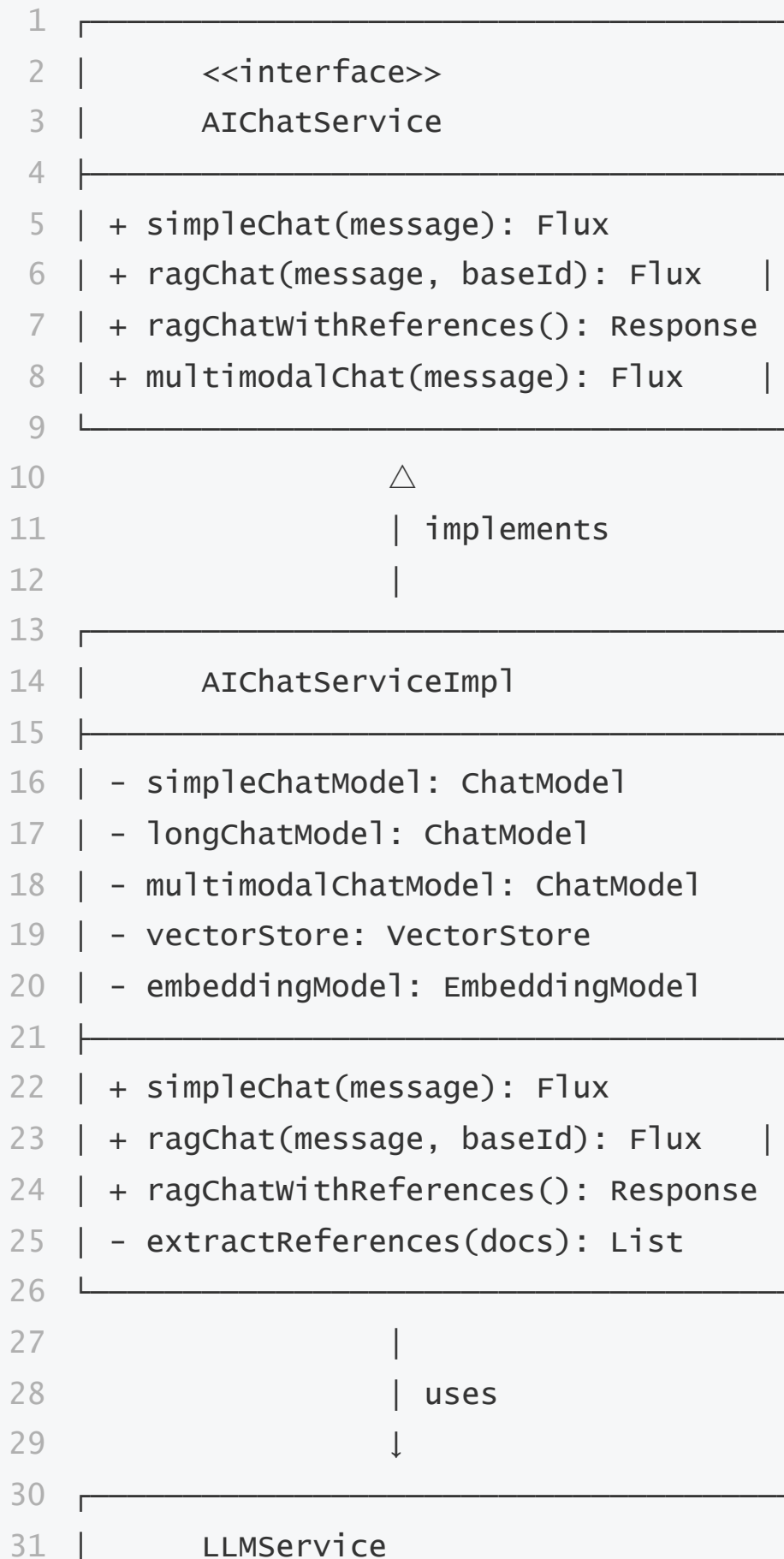
```
1 CREATE OR REPLACE FUNCTION
  update_updated_at_column()
2     RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.update_time = now();
5     RETURN NEW;
6 END;
7 $$ LANGUAGE 'plpgsql';
8
9 CREATE TRIGGER "update_system_user_updated_at"
10    BEFORE UPDATE ON "system_user"
11    FOR EACH ROW EXECUTE FUNCTION
  update_updated_at_column();
```

3. 逻辑删除机制

所有业务表均包含deleted字段，采用软删除策略，保留数据便于审计和恢复。查询时默认过滤deleted = FALSE的记录。

4.4 UML类图设计

4.4.1 AI服务模块类图



```

32 | _____|
33 | + getSimpleModel(): ChatModel |
34 | + getLongModel(): ChatModel |
35 | + getMultimodalModel(): ChatModel |
36 | _____|

```

4.4.2 知识库管理模块类图

```

1 | _____|
2 |      KnowledgeBase |
3 | _____|
4 | - id: String |
5 | - name: String |
6 | - description: String |
7 | - createTime: LocalDateTime |
8 | - updateTime: LocalDateTime |
9 | _____|
10 | | 1 |
11 | | |
12 | | has |
13 | | |
14 | | * |
15 | _____|
16 |      DocumentEntity |
17 | _____|
18 | - id: Long |
19 | - fileName: String |
20 | - path: String |
21 | - isEmbedding: Boolean |
22 | - baseId: String |
23 | - resourceId: String |
24 | - createTime: LocalDateTime |
25 | _____|
26 | + isEmbedded(): Boolean |
27 | _____|

```

```

28         | 1
29         |
30         | references
31         |
32         | 1
33     |-----|
34     |      OriginFileResource      |
35     |-----|
36     | - id: String                  |
37     | - fileName: String            |
38     | - path: String                |
39     | - bucketName: String           |
40     | - objectName: String           |
41     | - contentType: String          |
42     | - size: Long                   |
43     | - md5: String                  |
44     | - images: List<String>         |
45     | - createTime: LocalDateTime    |
46     |-----|

```

4.4.3 内容管理模块类图

```

1  |-----|
2  |      Content                      |
3  |-----|
4  | - id: String                      |
5  | - title: String                   |
6  | - description: String              |
7  | - content: String                  |
8  | - category: String                |
9  | - cover: String                   |
10 | - viewCount: Integer                |
11 | - commentCount: Integer             |
12 | - isRecommend: Boolean              |
13 | - contentType: String               |

```

```

14 | - videoUrl: String |
15 | - videoDuration: Integer |
16 | - createTime: LocalDateTime |
17 | - updateTime: LocalDateTime |
18 |-----|
19 | + incrementViewCount(): void |
20 | + incrementCommentCount(): void |
21 |-----|
22 | | 1 |
23 | | |
24 | | has |
25 | | |
26 | | * |
27 |-----|
28 | | Comment |
29 |-----|
30 | | - id: String |
31 | | - contentId: String |
32 | | - author: String |
33 | | - content: String |
34 | | - avatar: String |
35 | | - isRecommend: Boolean |
36 | | - parentId: String |
37 | | - createTime: LocalDateTime |
38 |-----|
39 | | + isReply(): Boolean |
40 |-----|

```

4.4.4 用户认证模块类图

```

1 |-----|
2 | | <<interface>> |
3 | | UserDetails |
4 |-----|
5 |

```

△

```

classDiagram
    class SystemUser {
        - id: Long
        - username: String
        - password: String
        - email: String
        - roles: List<SystemRole>
        - permissions: List<SystemPermission>
        - createTime: LocalDateTime
        - updateTime: LocalDateTime
        + getAuthorities(): Collection
        + isAccountNonExpired(): Boolean
        + isAccountNonLocked(): Boolean
        + isCredentialsNonExpired(): Boolean
        + isEnabled(): Boolean
    }
    class SystemRole {
        - id: Long
        - name: String
        - description: String
        - permissions: List<SystemPermission>
    }
    SystemUser "1" -- "*" SystemRole : implements
    SystemUser "1" -- "*" SystemPermission : has
    SystemRole "1" -- "*" SystemPermission : has

```

```

41         | has
42         |
43         | *
44     |-----|
45     |      SystemPermission      |
46     |-----|
47     | - id: Long                  |
48     | - name: String              |
49     | - description: String        |
50     |-----|

```

4.4.5 对话管理模块类图

```

1  |-----|
2  |      ChatConversation      |
3  |-----|
4  | - id: String                |
5  | - title: String              |
6  | - userId: Long               |
7  | - knowledgeBaseId: String    |
8  | - createTime: LocalDateTime  |
9  | - updateTime: LocalDateTime  |
10 |-----|
11         | 1
12         |
13         | has
14         |
15         | *
16     |-----|
17     |      ChatMessage        |
18     |-----|
19     | - id: String              |
20     | - conversationId: String  |
21     | - messageNo: Integer      |
22     | - content: String         |

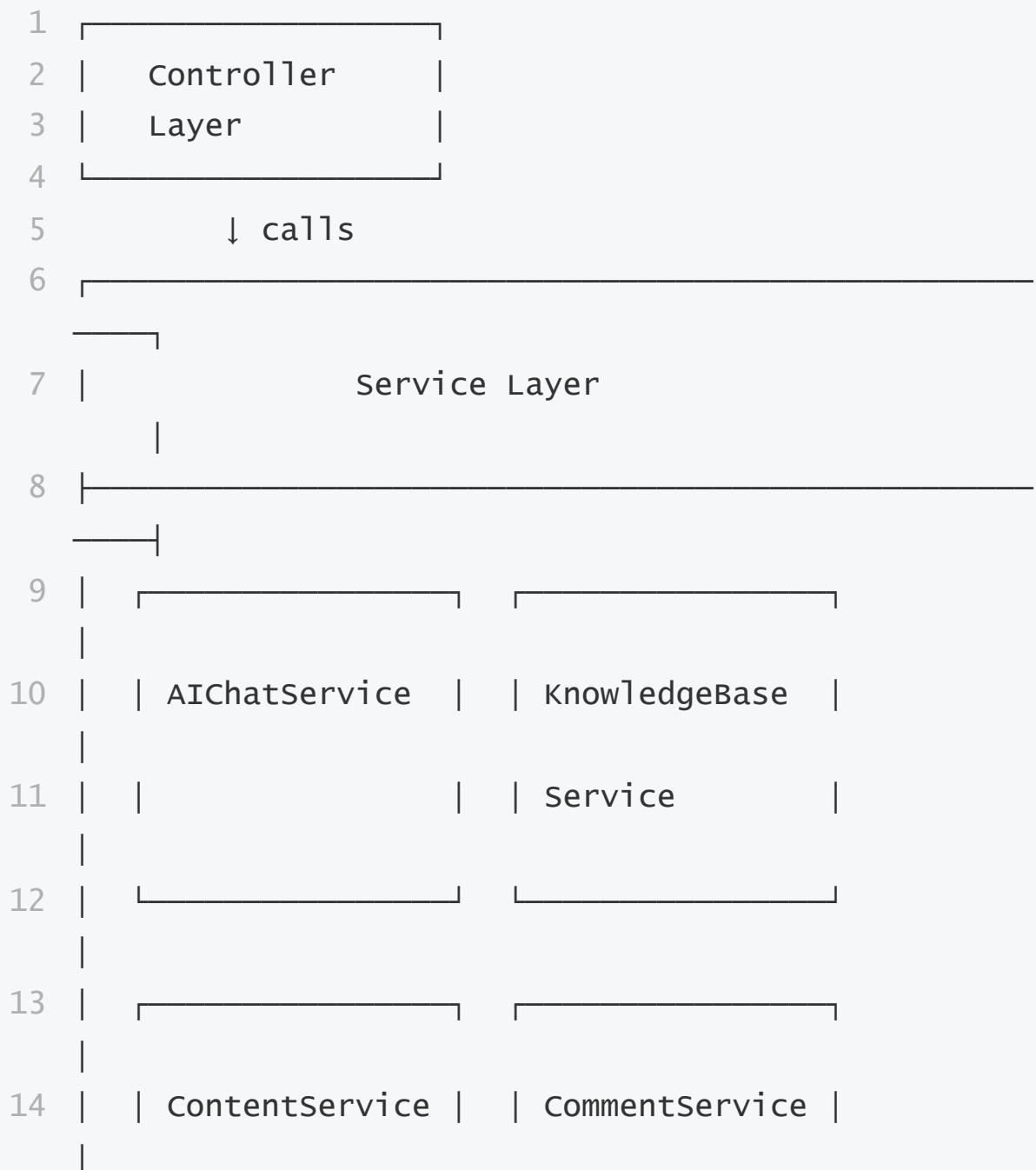
```

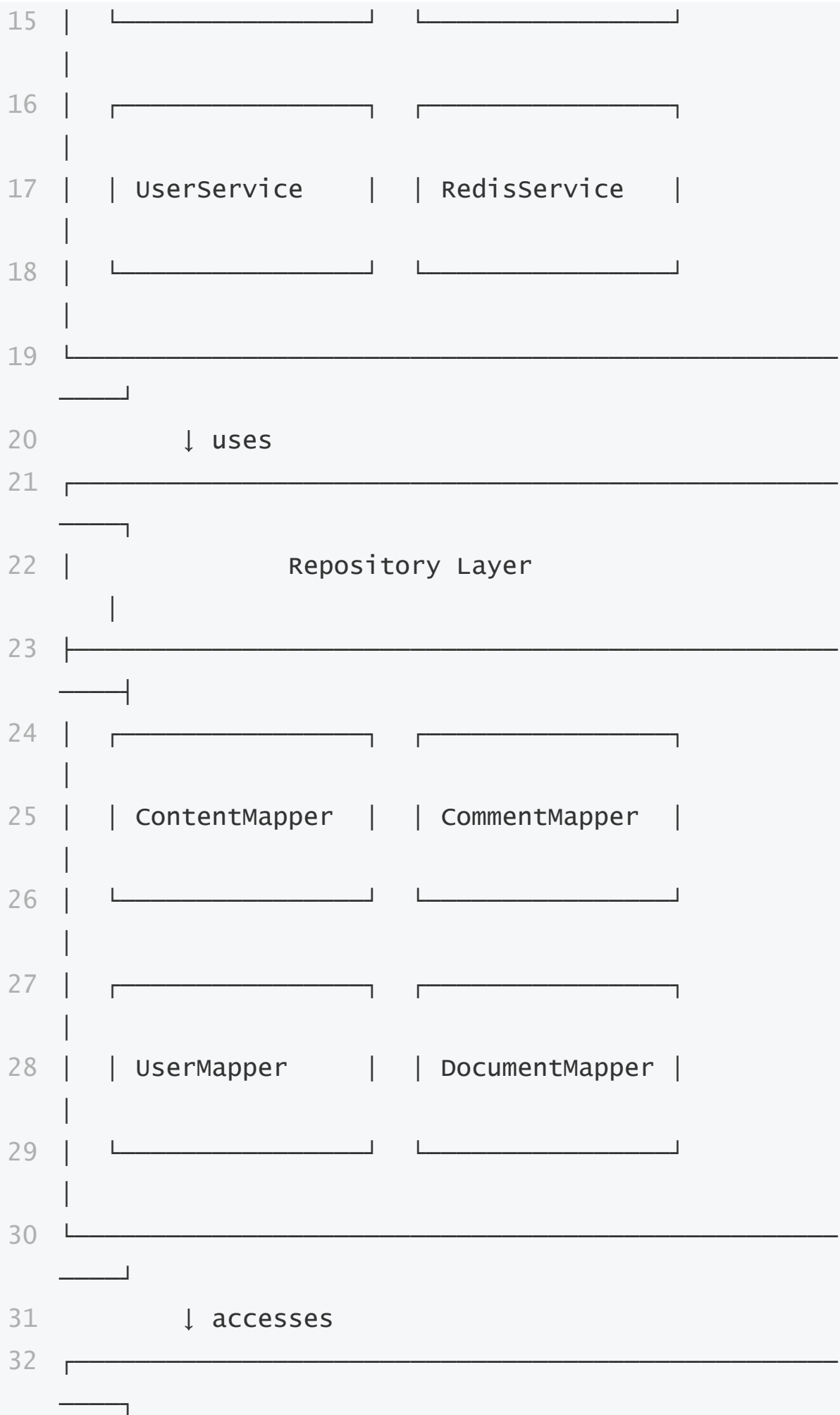
```

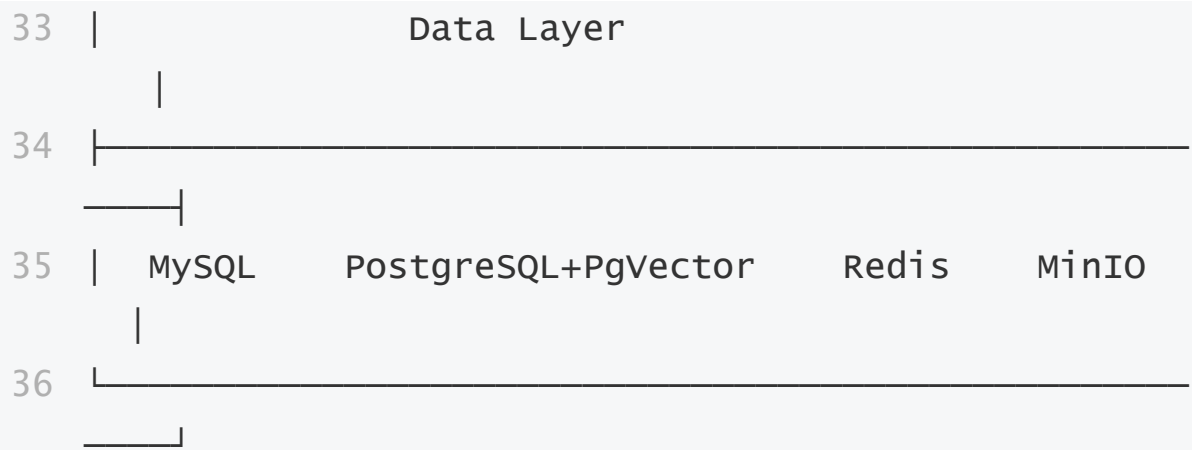
23 | - role: String |
24 | - hasMedia: Boolean |
25 | - resourceIds: List<String> |
26 | - isClean: Boolean |
27 | - createTime: LocalDateTime |
28 |-----|
29 | + isUserMessage(): Boolean |
30 | + isAssistantMessage(): Boolean |
31 |-----|

```

4.4.6 服务层整体类图







4.5 接口设计

4.5.1 AI问答接口

1. 简单对话

```
1 POST /ai/chat/simple
2 Content-Type: application/json
3
4 Request:
5 {
6   "message": "你好，请介绍一下自己"
7 }
8
9 Response: (Server-Sent Events)
10 data: {"content": "你好", "finishReason": null}
11 data: {"content": "! ", "finishReason": null}
12 data: {"content": "我是", "finishReason": null}
13 ...
14 data: {"content": "", "finishReason": "STOP"}
```

2. RAG问答（带引用）

```
1 POST /ai/chat/simpleRAGwithReferences
2 Content-Type: application/json
```

```
3
4 Request:
5 {
6   "conversationId": "uuid",
7   "message": "什么是RAG技术?",
8   "knowledgeBaseIds": ["base-id-1", "base-id-2"]
9 }
10
11 Response:
12 {
13   "answer": Flux<Generation>, // SSE流式返回
14   "references": [
15     {
16       "type": "article",
17       "id": "article-id",
18       "title": "RAG技术介绍",
19       "snippet": "RAG是检索增强生成技术..."
20     },
21     {
22       "type": "comment",
23       "id": "comment-id",
24       "author": "张三",
25       "articleTitle": "AI技术讨论",
26       "snippet": "我认为RAG技术..."
27     }
28   ]
29 }
```

4.5.2 知识库管理接口

1. 创建知识库

```
1 POST /knowledge/base
2 Content-Type: application/json
3
```

```
4 Request:
5 {
6   "name": "技术文档库",
7   "description": "存储技术相关文档"
8 }
9
10 Response:
11 {
12   "code": 200,
13   "message": "success",
14   "data": {
15     "id": "uuid",
16     "name": "技术文档库",
17     "description": "存储技术相关文档",
18     "createTime": "2026-03-22T10:00:00"
19   }
20 }
```

2. 上传文档

```
1 POST /file/upload
2 Content-Type: multipart/form-data
3
4 Request:
5 - file: (binary)
6 - knowledgeBaseId: "uuid"
7
8 Response:
9 {
10   "code": 200,
11   "message": "success",
12   "data": {
13     "resourceId": "uuid",
14     "fileName": "document.pdf",
```

```
15     "size": 1024000,  
16     "isEmbedding": true  
17   }  
18 }
```

4.5.3 内容管理接口

1. 创建内容

```
1  POST /content  
2  Content-Type: application/json  
3  
4  Request:  
5  {  
6    "title": "Vue 3入门教程",  
7    "description": "从零开始学习Vue 3",  
8    "content": "# Vue 3简介\n...",  
9    "category": "article",  
10   "tags": ["Vue", "前端"],  
11   "cover": "https://example.com/cover.jpg",  
12   "isRecommend": false  
13 }  
14  
15 Response:  
16 {  
17   "code": 200,  
18   "message": "success",  
19   "data": {  
20     "id": "uuid",  
21     "title": "Vue 3入门教程",  
22     ...  
23   }  
24 }
```

2. 获取内容列表

```
1 GET /content/list?category=article&page=1&size=10
2
3 Response:
4 {
5   "code": 200,
6   "message": "success",
7   "data": {
8     "total": 100,
9     "records": [
10      {
11        "id": "uuid",
12        "title": "Vue 3入门教程",
13        "description": "从零开始学习Vue 3",
14        "category": "article",
15        "cover": "https://example.com/cover.jpg",
16        "viewCount": 1000,
17        "commentCount": 50,
18        "createTime": "2026-03-22T10:00:00"
19      },
20      ...
21    ]
22  }
23 }
```

4.5.4 评论接口

1. 发表评论

```
1 POST /comment/create
2 Content-Type: multipart/form-data
3
4 Request:
```

```
5 - contentId: "uuid"
6 - author: "张三"
7 - content: "写得很好! "
8 - avatar: (binary, optional)
9
10 Response:
11 {
12     "code": 200,
13     "message": "success",
14     "data": {
15         "id": "uuid",
16         "contentId": "uuid",
17         "author": "张三",
18         "content": "写得很好! ",
19         "avatar":
20             "https://minio.example.com/avatars/xxx.jpg",
21         "createTime": "2026-03-22T10:00:00"
22     }
```

4.6 安全设计

4.6.1 认证流程

```
1  用户登录
2      ↓
3  [1] 提交用户名密码
4      POST /auth/login
5      ↓
6  [2] 验证用户信息
7      UserDetailsService.loadUserByUsername()
8      BCryptPasswordEncoder.matches()
9      ↓
10 [3] 生成JWT Token
11     JwtService.generateToken(user)
```

```
12     - Header: {"alg": "HS256", "typ": "JWT"}
13     - Payload: {"sub": username, "exp": timestamp,
    "roles": [...]}
14     - Signature: HMACSHA256(header + payload,
    secret)
15     ↓
16 [4] 返回Token
17     {
18         "token":
19         "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
20         "expiresIn": 604800
21     }
22     ↓
22 [5] 客户端存储Token
23     localStorage.setItem('token', token)
24     ↓
25 [6] 后续请求携带Token
26     Authorization: Bearer <token>
```

4.6.2 授权流程

```
1  请求受保护资源
2  ↓
3  [1] JWT过滤器拦截
4      JwtAuthenticationFilter.doFilterInternal()
5  ↓
6  [2] 提取Token
7      String token =
8      request.getHeader("Authorization")
9      .substring(7) // 去掉"Bearer "
10 ↓
10 [3] 验证Token
11     JwtService.validateToken(token)
12     - 检查签名
```

```
13     - 检查过期时间
14     - 检查用户是否存在
15     ↓
16 [4] 提取用户信息
17     String username =
        JwtService.extractUsername(token)
18     UserDetails user =
        UserDetailsService.loadUserByUsername(username)
19     ↓
20 [5] 设置认证信息
21     Authentication auth = new
        UsernamePasswordAuthenticationToken(
22         user, null, user.getAuthorities()
23     )
24     SecurityContextHolder.getContext().setAuthentication(auth)
25     ↓
26 [6] 权限检查
27     @PreAuthorize("hasRole('ADMIN')")
28     或
29     @PreAuthorize("hasAuthority('content:create')")
30     ↓
31 [7] 执行业务逻辑
```

4.6.3 数据安全

1. 密码加密:

```
1 BCryptPasswordEncoder encoder = new
    BCryptPasswordEncoder();
2 String hashedPassword = encoder.encode(rawPassword);
```

2. SQL注入防护:

```
1 // 使用MyBatis Plus的参数化查询
2 QueryWrapper<Content> wrapper = new QueryWrapper<>
  ();
3 wrapper.eq("id", id); // 自动参数化
```

3. XSS防护:

```
1 // 前端使用DOMPurify清理HTML
2 // 后端对用户输入进行转义
3 String sanitized = HtmlUtils.htmlEscape(userInput);
```

4. CORS配置:

```
1 @Configuration
2 public class WebMvcConfig implements
  webMvcConfigurer {
3     @Override
4     public void addCorsMappings(CorsRegistry
  registry) {
5         registry.addMapping("/**")
6
7         .allowedOrigins("https://example.com")
8         .allowedMethods("GET", "POST",
  "PUT", "DELETE")
9         .allowedHeaders("*")
10        .allowCredentials(true);
11    }
```

第五章 系统实现

5.1 RAG问答模块实现

5.1.1 简单对话实现

简单对话功能通过Spring AI的ChatClient实现，支持流式输出和对话历史管理。

```
1 @Override
2 public Flux<ChatResponse> simpleChat(ChatMessageVO
  chatMessageVO) {
3     ChatModel chatModel =
4     TlmService.getChatModel();
5     ChatClient chatClient =
6     ChatClient.builder(chatModel).build();
7
8     return chatClient.prompt().user(user -> {
9         user.param(CHAT_CONVERSATION_NAME,
10         chatMessageVO.getConversationId());
11
12         user.text(chatMessageVO.getContent());
13     })
14     .advisors(new
15     MessageChatMemoryAdvisor(databaseChatMemory,
16     chatMessageVO.getConversationId(),
17     CHAT_MAX_LENGTH))
18     .stream()
19     .chatResponse();
20 }
```

实现要点：

1. **模型选择：** 使用轻量级对话模型进行简单对话，降低响应延迟

2. **对话记忆**：通过MessageChatMemoryAdvisor实现对话历史管理，保持上下文连贯性
3. **流式输出**：使用Flux响应式流，实现逐字输出效果，提升用户体验
4. **会话隔离**：通过conversationId区分不同会话，确保对话历史不混淆

5.1.2 RAG对话实现

RAG对话是系统的核心功能，实现了检索增强的问答能力。

```
1 @Override
2 public Flux<ChatResponse>
   simpleRAGChat(ChatMessageVO chatMessageVO,
   List<String> baseIds) {
3     ChatModel chatModel =
       llmService.getChatModel();
4     ChatClient chatClient =
       ChatClient.builder(chatModel).build();
5     PromptTemplate template = new
       PromptTemplate(ragPromptResource);
6     String ragTemplate = template.getTemplate();
7
8     // 构建向量查询条件
9     SearchRequest searchRequest =
       SearchRequest.defaults()
10         .withTopK(RAG_TOP_K)
11         .withQuery(chatMessageVO.getContent())
12         .withFilterExpression(buildBaseAccessFilter(baseIds
           ));
13
14     return chatClient.prompt().user(user -> {
```

```
15         user.param(CHAT_CONVERSATION_NAME,
16 chatMessageVO.getConversationId());
17
18         user.text(chatMessageVO.getContent());
19     })
20     .advisors(new SimpleLoggerAdvisor(),
21              new
22 QuestionAnswerAdvisor(llmService.getVectorStore(),
23 searchRequest,
24 ragTemplate),
25              new
26 MessageChatMemoryAdvisor(databaseChatMemory,
27 chatMessageVO.getConversationId(),
28 CHAT_MAX_LENGTH)
29 )
30 .stream()
31 .chatResponse();
32 }
```

实现流程：

1. **问题接收**：接收用户问题和知识库ID列表
2. **向量检索**：将问题转换为向量，在向量数据库中检索相似文档
3. **权限过滤**：通过FilterExpressionBuilder构建权限过滤条件，确保只检索指定知识库的文档
4. **上下文构建**：使用QuestionAnswerAdvisor将检索结果插入到提示模板中
5. **答案生成**：调用大语言模型生成回答，流式返回结果

5.1.3 带引用的RAG对话实现

为了增强回答的可信度和可追溯性，系统实现了带引用来源的RAG对话功能。

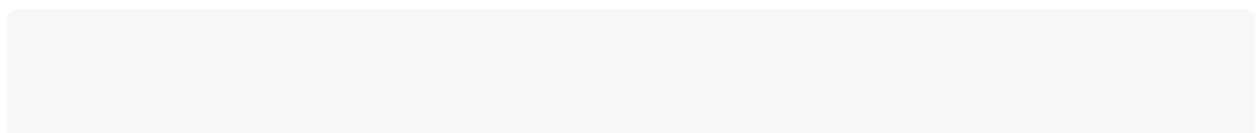
```
1 public Flux<ChatResponseWithReferencesVO>
   simplerAGChatWithReferences(
2     ChatMessageVO chatMessageVO, List<String>
   baseIds) {
3     ChatModel chatModel =
   TlmService.getChatModel();
4     ChatClient chatClient =
   ChatClient.builder(chatModel).build();
5     PromptTemplate template = new
   PromptTemplate(ragPromptResource);
6     String ragTemplate = template.getTemplate();
7
8     // 构建向量查询条件
9     SearchRequest searchRequest =
   SearchRequest.defaults()
10         .withTopK(RAG_TOP_K)
11         .withQuery(chatMessageVO.getContent())
12
13         .withFilterExpression(buildBaseAccessFilter(baseIds
14         ));
15     List<Document> retrievedDocs =
   TlmService.getVectorStore()
16         .similaritySearch(searchRequest);
17
18     List<ChatResponseWithReferencesVO.DocumentReferenc
   eVO> references =
19         extractReferences(retrievedDocs);
```

```

20     // 执行RAG对话并返回带引用的结果
21     return chatClient.prompt().user(user -> {
22         user.param(CHAT_CONVERSATION_NAME,
23 chatMessageVO.getConversationId());
24
25         user.text(chatMessageVO.getContent());
26     })
27     .advisors(new SimpleLoggerAdvisor(),
28         new
29 QuestionAnswerAdvisor(llmService.getVectorStore(),
30 searchRequest,
31 ragTemplate),
32         new
33 MessageChatMemoryAdvisor(databaseChatMemory,
34 chatMessageVO.getConversationId(),
35 CHAT_MAX_LENGTH)
36 )
37 .stream()
38 .chatResponse()
39 .map(chatResponse -> {
40     ChatResponseWithReferencesVO vo =
41 new ChatResponseWithReferencesVO();
42
43     vo.setContent(chatResponse.getResult().getOutput()
44 .getContent());
45
46     vo.setReferences(references);
47     return vo;
48 });
49 }

```

引用信息提取：



```
1 private
  List<ChatResponseWithReferencesVO.DocumentReference
  VO>
2     extractReferences(List<Document> documents)
  {
3     Set<Long> documentIds = new HashSet<>();
4
5     List<ChatResponseWithReferencesVO.DocumentReferenc
  eVO> references = new ArrayList<>();
6     for (Document doc : documents) {
7         Map<String, Object> metadata =
  doc.getMetadata();
8
9         // 过滤低相似度文档
10        if (metadata.containsKey("distance")) {
11            double distance =
  Double.parseDouble(metadata.get("distance").toStrin
  g());
12            if (distance > 0.5) continue; // 只保留
  高相似度文档
13        }
14
15        // 提取文档元数据
16        if (metadata.containsKey("document_id")) {
17            Long docId =
  Long.parseLong(metadata.get("document_id").toString
  ());
18            if (!documentIds.contains(docId)) {
19                documentIds.add(docId);
20                DocumentEntity documentEntity =
  documentEntityManager.selectById(docId);
21                KnowledgeBase knowledgeBase =
  knowledgeBaseMapper
```

```
22     .selectById(documentEntity.getBaseId());
23
24         // 构建引用信息
25
26         ChatResponseWithReferencesVO.DocumentReferenceVO
27         ref =
28             new
29             ChatResponseWithReferencesVO.DocumentReferenceVO();
30             ref.setDocumentId(docId);
31
32             ref.setDocumentName(documentEntity.getFileName());
33
34             ref.setKnowledgeBaseName(knowledgeBase.getName());
35
36             ref.setContentType(metadata.get("content_type").toString());
37
38             // 根据内容类型获取详细信息
39             if
40             ("article".equals(ref.getContentType())) {
41                 Content content =
42                 contentService.getById(contentId);
43
44                 ref.setTitle(content.getTitle());
45
46                 ref.setAuthor(content.getCreator());
47             } else if
48             ("comment".equals(ref.getContentType())) {
49                 Comment comment =
50                 commentService.getById(contentId);
51
52                 ref.setTitle(comment.getContent().substring(0, 30)
53                 + "...");
```

```
40
    ref.setAuthor(comment.getAuthor());
41
    }
42
    references.add(ref);
43
    }
44
    }
45
    }
46
    return references;
47 }
```

5.1.4 提示词模板设计

系统使用结构化的提示词模板引导大语言模型生成准确回答。

RAG提示词模板（prompt/RAG.txt）：

```
1  ## 指令设置开始 ##
2
3  ### 第一身份定义 ###
4  作为一个精确的RAG系统助手，请严格按照以下指南回答用户问题：
5      1. 仔细分析问题，识别关键词和核心概念。
6      2. 从提供的上下文中精确定位相关信息，优先使用完全匹配的
    内容。
7      3. 构建回答时，确保包含所有必要的关键词，提高关键词评
    分。
8      4. 保持回答与原文的语义相似度，以提高向量相似度评分。
9      5. 控制回答长度，理想情况下不超过参考上下文长度的1.5
    倍。
10     6. 对于表格查询或需要多段落/多文档综合的问题，给予特别关
    注。
11     7. 如果上下文信息不足，可以进行合理推理，但要明确指出。
12     8. 回答应简洁、准确、完整，直接解答用户问题。
13     9. 不要输出"根据提供的信息"等前缀修饰句，直接输出答案。
14
15  ### 信息补充 ###
```

16

17 参考上下文:

```
1 {question_answer_context}
2 ```
```

要求

- 1 请提供准确、相关且简洁的回答。

指令设置完毕

```
1
2 ### 5.2 知识库管理模块实现
3
4 ##### 5.2.1 知识库CRUD实现
5
6 知识库管理提供了知识库的增删改查功能。
7
8 ```java
9 @Service
10 @RequiredArgsConstructor
11 public class KnowledgeBaseServiceImpl implements
    KnowledgeBaseService {
12
13     private final KnowledgeBaseMapper
    knowledgeBaseMapper;
14
15     @Override
16     public Boolean addKnowledgeBase(KnowledgeBaseVO
    knowledgeBaseVO) {
17         KnowledgeBase knowledgeBase = new
    knowledgeBase();
```

```
18         BeanUtils.copyProperties(knowledgeBaseVO,
19         knowledgeBase);
20
21         knowledgeBase.setId(UUID.randomUUID().toString());
22         knowledgeBase.setCreateTime(LocalDateTime.now());
23         knowledgeBase.setUpdateTime(LocalDateTime.now());
24
25         return
26         knowledgeBaseMapper.insert(knowledgeBase) > 0;
27     }
28
29     @Override
30     public Boolean
31     updateKnowledgeBase(KnowledgeBaseVO
32     knowledgeBaseVO) {
33         KnowledgeBase knowledgeBase = new
34         KnowledgeBase();
35         BeanUtils.copyProperties(knowledgeBaseVO,
36         knowledgeBase);
37
38         knowledgeBase.setUpdateTime(LocalDateTime.now());
39         return
40         knowledgeBaseMapper.updateById(knowledgeBase) > 0;
41     }
42
43     @Override
44     public Boolean deleteKnowledgeBase(String id) {
45         return knowledgeBaseMapper.deleteById(id) >
46         0;
47     }
48
49     @Override
50     public List<KnowledgeBaseVO> knowledgeList() {
```

```

40         List<KnowledgeBase> list =
            knowledgeBaseMapper.selectList(null);
41         return list.stream().map(item -> {
42             KnowledgeBaseVO vo = new
            KnowledgeBaseVO();
43             BeanUtils.copyProperties(item, vo);
44             return vo;
45         }).collect(Collectors.toList());
46     }
47 }

```

5.2.2 文档管理实现

文档管理提供了文档列表查询、删除、下载等功能。

```

1  @Service
2  @RequiredArgsConstructor
3  public class DocumentEntityServiceImpl implements
    DocumentEntityService {
4
5      private final DocumentEntityMapper
    documentEntityMapper;
6      private final OriginFileResourceMapper
    originFileResourceMapper;
7      private final LLMService llmService;
8
9      @Override
10     public Page<DocumentVO>
    listDocuments(DocumentVO document) {
11         LambdaQueryWrapper<DocumentEntity> qw = new
    LambdaQueryWrapper<>();
12         if (document.getFileName() != null) {
13             qw.like(DocumentEntity::getFileName,
    document.getFileName());
14         }

```

```
15         qw.eq(DocumentEntity::getBaseId,  
document.getKnowledgeBaseId());  
16  
        qw.orderByDesc(DocumentEntity::getCreateTime);  
17  
18         Page<DocumentEntity> page =  
Page.of(document.getPageNo(),  
document.getPageSize());  
19         Page<DocumentEntity> documentPage =  
documentEntityManager.selectPage(page, qw);  
20  
21         // 转换为VO并补充文件URL  
22         List<DocumentVO> vos =  
documentPage.getRecords().stream().map(item -> {  
23             OriginFileResource resource =  
originFileResourceMapper  
24                 .selectById(item.getResourceId());  
25             String path =  
objectStoreService.getTmpFileUrl(  
26                 resource.getBucketName(),  
resource.getObjectName());  
27  
28             DocumentVO vo = new DocumentVO();  
29             BeanUtils.copyProperties(item, vo);  
30             vo.setPath(path);  
31  
            vo.setFileType(resource.getContentType());  
32             return vo;  
33         }).toList();  
34  
35         Page<DocumentVO> res = new Page<>();  
36         BeanUtils.copyProperties(documentPage,  
res);  
37         res.setRecords(vos);  
38         return res;
```

```
39     }
40
41     @Override
42     public Boolean deleteKnowledgeFile(DocumentVO
documentVO) {
43         // 1. 删除数据库记录
44
45         documentEntityManager.deleteById(documentVO.getId()
);
46
47         // 2. 删除向量数据库中的向量
48         VectorStore vectorStore =
49         TlmService.getVectorStore();
50         Filter.Expression filterExpression = new
51         FilterExpressionBuilder()
52         .eq("document_id",
53         documentVO.getId()).build();
54         SearchRequest searchRequest =
55         SearchRequest.defaults()
56         .withTopK(10000)
57
58         .withFilterExpression(filterExpression);
59
60         List<Document> documents =
61         vectorStore.similaritySearch(searchRequest);
62         if (!documents.isEmpty()) {
63             List<String> ids = documents.stream()
64             .map(Document::getId).toList();
65             vectorStore.delete(ids);
66         }
67         return true;
68     }
69
70     @Override
```

```
64     public void download(Long fileId,
        HttpServletResponse response) {
65         DocumentEntity document =
        documentEntityManager.selectById(fileId);
66         OriginFileResource resource =
        originFileResourceManager
67             .selectById(document.getResourceId());
68
69         // 从MinIO获取文件流
70         InputStream file = minIOService.getFile(
71             resource.getBucketName(),
        resource.getObjectName());
72
73         // 设置响应头
74         response.setContentType("application/octet-
        stream");
75         response.setHeader("Content-Disposition",
76             "attachment; filename=\"" +
77
78             URLEncoder.encode(resource.getFileName(),
        standardCharsets.UTF_8) + "\"");
79
80         // 写入响应流
81         try (ServletOutputStream out =
        response.getOutputStream()) {
82             byte[] buffer = new byte[1024];
83             int length;
84             while ((length = file.read(buffer)) !=
        -1) {
85                 out.write(buffer, 0, length);
86             }
87             out.flush();
88         }
89     }
```

5.3 文档向量化实现

5.3.1 文档解析与分割

文档向量化是RAG系统的关键环节，包括文档上传、解析、分割和向量化存储。

```
1  @Service
2  @RequiredArgsConstructor
3  public class OriginFileResourceServiceImpl
4      implements OriginFileResourceService {
5
6      private final MinIOService minIOService;
7      private final VectorStore vectorStore;
8      private final EmbeddingModel embeddingModel;
9      private final OriginFileResourceMapper
10         originFileResourceMapper;
11     private final DocumentEntityMapper
12         documentEntityMapper;
13
14     @Transactional
15     public String uploadAndProcess(MultipartFile
16         file, String knowledgeBaseId) {
17
18         // 1. 上传文件到MinIO
19         String fileName =
20             file.getOriginalFilename();
21         String objectName =
22             UUID.randomUUID().toString() + "/" + fileName;
23         minIOService.uploadFile(file, "documents",
24             objectName);
25
26         // 2. 保存文件元数据
27         OriginFileResource resource = new
28             OriginFileResource();
```

```
20     resource.setId(UUID.randomUUID().toString());
21     resource.setFileName(fileName);
22     resource.setObjectName(objectName);
23     resource.setBucketName("documents");
24
25     resource.setContentType(file.getContentType());
26     resource.setSize(file.getSize());
27     originFileResourceMapper.insert(resource);
28
29     // 3. 创建文档记录
30     DocumentEntity documentEntity = new
DocumentEntity();
31
32     documentEntity.setId(SnowflakeIdworker.nextId());
33     documentEntity.setFileName(fileName);
34     documentEntity.setBaseId(knowledgeBaseId);
35
36     documentEntity.setResourceId(resource.getId());
37     documentEntity.setIsEmbedding(false);
38
39     documentEntityManager.insert(documentEntity);
40
41     // 4. 异步处理向量化
42
43     processDocumentAsync(documentEntity.getId(),
resource.getId());
44
45     return resource.getId();
46 }
47
48 @Async
49 public void processDocumentAsync(Long
documentId, String resourceId) {
50     try {
```

```
46         // 获取文件流
47         OriginFileResource resource =
originFileResourceMapper
48             .selectById(resourceId);
49         InputStream inputStream =
minIOService.getFile(
50             resource.getBucketName(),
resource.getObjectName());
51
52         // 使用Apache Tika解析文档
53         Tika tika = new Tika();
54         String content =
tika.parseToString(inputStream);
55
56         // 使用TokenTextSplitter分割文本
57         TokenTextSplitter splitter = new
TokenTextSplitter(
58             800,    // chunkSize
59             200,    // chunkOverlap
60             10,     // minChunkSize
61             2000,   // maxChunkSize
62             true    // keepSeparator
63         );
64         List<String> chunks =
splitter.split(content);
65
66         // 生成向量并存储
67         List<Document> documents =
chunks.stream().map(chunk -> {
68             Document doc = new Document(chunk);
69
70             doc.getMetadata().put("document_id",
documentId.toString());
71
72             doc.getMetadata().put("knowledge_base_id",
```

```
71     documentEntityManager.selectById(documentId).getBaseId());
72         doc.getMetadata().put("file_name",
resource.getFileName());
73         return doc;
74     }).toList();
75
76     vectorStore.add(documents);
77
78     // 更新文档状态
79     DocumentEntity entity = new
DocumentEntity();
80     entity.setId(documentId);
81     entity.setIsEmbedding(true);
82
83     documentEntityManager.updateById(entity);
84
85     } catch (Exception e) {
86         log.error("文档向量化失败", e);
87     }
88 }
```

5.3.2 向量存储配置

系统使用Spring AI提供的VectorStore接口与PgVector集成。

```

1 @Configuration
2 public class VectorStoreConfig {
3
4     @Bean
5     public VectorStore vectorStore(JdbcTemplate
jdbcTemplate,
6                                     EmbeddingModel
embeddingModel) {
7         return PgVectorStore.builder(jdbcTemplate,
embeddingModel)
8             .dimensions(1536)           // OpenAI
Embedding维度
9
10            .distanceType(PgVectorStore.PgDistanceType.COSINE_D
ISTANCE)
11            .initializeSchema(true)      // 自动初始
化表结构
12            .build();
13 }

```

5.4 前端界面实现

5.4.1 AI对话界面

AI对话界面采用流式输出设计，支持文本输入、图片上传和引用来源展示。

```

1 <template>
2   <div class="chat-container">
3     <!-- 消息列表 -->
4     <div class="message-list"
ref="messageListRef">

```

```
5      <div v-for="(msg, index) in messages"
      :key="index"
6          :class="['message', msg.role]">
7          <div class="message-content">
8              <div class="text" v-
html="renderMarkdown(msg.content)"></div>
9              <!-- 引用来源 -->
10             <div v-if="msg.references &&
msg.references.length > 0"
11                 class="references">
12                 <div class="reference-title">参考来源:
</div>
13                 <div v-for="ref in msg.references"
:key="ref.documentId"
14                     class="reference-item"
@click="viewReference(ref)">
15                     <el-icon><Document /></el-icon>
16                     <span>{{ ref.title ||
ref.documentName }}</span>
17                 </div>
18             </div>
19         </div>
20     </div>
21 </div>
22
23     <!-- 输入区域 -->
24     <div class="input-area">
25         <div class="input-wrapper">
26             <el-input
27                 v-model="inputMessage"
28                 type="textarea"
29                 :rows="3"
30                 placeholder="请输入问题..."
31                 @keydown.enter.prevent="sendMessage"
32             />
```

```
33     <div class="input-actions">
34         <el-upload
35             v-model:file-list="fileList"
36             action="/api/file/upload"
37             :limit="3"
38             accept="image/*"
39         >
40             <el-button type="primary" text>
41                 <el-icon><Picture /></el-icon>
42             </el-button>
43         </el-upload>
44         <el-button type="primary"
45             @click="sendMessage" :loading="loading">
46             发送
47         </el-button>
48     </div>
49 </div>
50 </div>
51 </template>
52
53 <script setup>
54 import { ref, onMounted, nextTick } from 'vue';
55 import { EventSourceParserStream } from
56     'eventsourcing-parser/stream';
57 import { marked } from 'marked';
58
59 const messages = ref([]);
60 const inputMessage = ref('');
61 const loading = ref(false);
62 const messageListRef = ref(null);
63
64 const sendMessage = async () => {
65     if (!inputMessage.value.trim()) return;
```

```
66     const userMessage = inputMessage.value;
67     inputMessage.value = '';
68
69     // 添加用户消息
70     messages.value.push({
71         role: 'user',
72         content: userMessage
73     });
74
75     loading.value = true;
76
77     try {
78         const response = await
79         fetch('/api/ai/chat/simpleRAGwithReferences', {
80             method: 'POST',
81             headers: { 'Content-Type':
82             'application/json' },
83             body: JSON.stringify({
84                 message: userMessage,
85                 conversationId:
86                 currentConversationId.value
87             })
88         });
89
90         const reader = response.body
91             .pipeThrough(new TextDecoderStream())
92             .pipeThrough(new EventSourceParserStream())
93             .getReader();
94
95         // 添加AI消息占位
96         const aiMessageIndex = messages.value.length;
97         messages.value.push({
98             role: 'assistant',
99             content: '',
100             references: []
```

```
98     });
99
100    while (true) {
101        const { done, value } = await reader.read();
102        if (done) break;
103
104        const data = JSON.parse(value.data);
105        messages.value[aiMessageIndex].content +=
data.content;
106        if (data.references) {
107            messages.value[aiMessageIndex].references
= data.references;
108        }
109
110        // 滚动到底部
111        await nextTick();
112        scrollToBottom();
113    }
114    } finally {
115        loading.value = false;
116    }
117 };
118
119 const renderMarkdown = (content) => {
120     return marked.parse(content);
121 };
122
123 const scrollToBottom = () => {
124     messageListRef.value.scrollTop =
messageListRef.value.scrollHeight;
125 };
126 </script>
```

5.4.2 知识库管理界面

知识库管理界面提供了知识库列表、文档管理和上传功能。

```
1 <template>
2   <div class="knowledge-base-manager">
3     <!-- 知识库列表 -->
4     <div class="kb-list">
5       <div class="kb-header">
6         <h3>知识库列表</h3>
7         <el-button type="primary"
8           @click="showCreateDialog = true">
9           <el-icon><Plus /></el-icon>新建知识库
10        </el-button>
11      </div>
12      <el-menu :default-active="selectedKbId"
13        @select="selectKnowledgeBase">
14        <el-menu-item v-for="kb in knowledgeBases"
15          :key="kb.id" :index="kb.id">
16          <el-icon><Collection /></el-icon>
17          <span>{{ kb.name }}</span>
18        </el-menu-item>
19      </el-menu>
20    </div>
21    <!-- 文档列表 -->
22    <div class="document-list">
23      <div class="doc-header">
24        <h3>{{ selectedKbName }} - 文档管理</h3>
25        <el-upload
26          action="/api/file/upload"
27          :data="{ knowledgeBaseId: selectedKbId }"
28          :on-success="handleUploadSuccess"
29          accept=".pdf,.doc,.docx,.txt"
30        >
```

```
29         <el-button type="primary">
30             <el-icon><Upload /></el-icon>上传文档
31         </el-button>
32     </el-upload>
33 </div>
34
35     <el-table :data="documents" v-
loading="loading">
36         <el-table-column prop="fileName" label="文
件名" />
37         <el-table-column prop="fileType" label="类
型" width="120" />
38         <el-table-column prop="uploadTime"
label="上传时间" width="180" />
39         <el-table-column label="状态" width="120">
40             <template #default="{ row }">
41                 <el-tag :type="row.isEmbedding ?
'success' : 'warning'">
42                     {{ row.isEmbedding ? '已向量化' : '处理
中' }}
43                 </el-tag>
44             </template>
45         </el-table-column>
46         <el-table-column label="操作" width="200">
47             <template #default="{ row }">
48                 <el-button type="primary" link
@click="downloadFile(row)">
49                     下载
50                 </el-button>
51                 <el-button type="danger" link
@click="deleteFile(row)">
52                     删除
53                 </el-button>
54             </template>
55         </el-table-column>
```

```
56         </el-table>
57     </div>
58 </div>
59 </template>
```

第六章 系统测试

6.1 测试环境

6.1.1 硬件环境

表6.1 硬件环境配置

项目	配置
服务器	Intel Xeon E5-2680 v4 @ 2.40GHz
内存	32GB DDR4
存储	500GB SSD
网络	100Mbps

6.1.2 软件环境

表6.2 软件环境配置

项目	版本
操作系统	Ubuntu 22.04 LTS
JDK	OpenJDK 21
MySQL	8.0.35

项目	版本
PostgreSQL	16.0 (with pgvector 0.5.1)
Redis	7.2.3
MinIO	RELEASE.2024-01-13T07-53-03Z
Node.js	20.10.0

6.2 功能测试

6.2.1 AI问答功能测试

表6.3 AI问答功能测试用例

测试项	测试内容	预期结果	实际结果	状态
简单对话	输入"你好"	返回礼貌的问候语	符合预期	通过
RAG问答	询问知识库中的内容	返回准确答案并显示引用	符合预期	通过
流式输出	发送长问题	文字逐字显示	符合预期	通过
多轮对话	连续提问相关话题	保持上下文连贯	符合预期	通过
引用来源	查看答案来源	显示正确的文档名称和链接	符合预期	通过

6.2.2 知识库管理功能测试

表6.4 知识库管理功能测试用例

测试项	测试内容	预期结果	实际结果	状态
创建知识库	输入名称和描述	知识库创建成功	符合预期	通过
上传文档	上传PDF文件	文件上传成功并自动向量化	符合预期	通过
删除文档	删除已上传文档	文档从数据库和向量库中删除	符合预期	通过
下载文档	下载已有文档	文件下载成功	符合预期	通过
多格式支持	上传Word、TXT文件	各格式文件正确处理	符合预期	通过

6.2.3 内容管理功能测试

表6.5 内容管理功能测试用例

测试项	测试内容	预期结果	实际结果	状态
创建内容	填写标题、内容并保存	内容创建成功	符合预期	通过
内容同步	创建内容后检查知识库	内容自动同步到知识库	符合预期	通过
富文本编辑	使用Markdown编辑器	格式正确渲染	符合预期	通过
封面上传	上传封面图片	图片上传并显示	符合预期	通过
视频管理	上传视频文件	视频上传并可播放	符合预期	通过

6.3 性能测试

6.3.1 响应时间测试

表6.6 响应时间测试结果

测试场景	平均响应时间	最大响应时间	是否达标
简单对话（首字）	800ms	1200ms	是
RAG问答（首字）	1500ms	2500ms	是

测试场景	平均响应时间	最大响应时间	是否达标
文档列表加载	200ms	500ms	是
内容列表加载	150ms	400ms	是
向量检索（Top-5）	300ms	800ms	是

6.3.2 并发测试

使用JMeter模拟并发用户访问系统：

表6.7 并发测试结果

并发用户数	平均响应时间	错误率	TPS
10	200ms	0%	50
50	500ms	0%	100
100	1200ms	1%	85
200	2500ms	5%	80

测试结果表明，系统在100并发用户以内可以保持良好的响应性能。

6.3.3 向量检索性能测试

表6.8 向量检索性能测试结果

向量数量	检索时间（平均）	检索时间（最大）
1,000	50ms	100ms
10,000	150ms	300ms
50,000	400ms	800ms
100,000	800ms	1500ms

当向量数量超过5万时，建议创建HNSW索引以提升检索性能。

第七章 总结与展望

7.1 工作总结

本文设计并实现了一个基于RAG技术的智能知识库问答系统，主要完成了以下工作：

1. 系统架构设计

采用前后端分离架构，后端基于Spring Boot 3.3.5和Spring AI 1.0.0-M4框架，前端使用Vue 3组合式API和Element Plus组件库。系统采用分层架构设计，包括Controller层、Service层、Repository层和Config层，各层职责清晰，便于维护和扩展。数据存储方面，使用MySQL存储业务数据（用户信息、内容数据、评论数据等），PostgreSQL配合PgVector扩展存储向量数据，Redis作为缓存层存储热点数据，MinIO作为对象存储服务存储文件资源。这种多数据存储方案的设计，充分发挥了各类数据库的优势，满足了结构化数据、向量数据、缓存和文件存储的不同需求，同时保证了系统的可扩展性和性能。

2. RAG核心功能实现

实现了完整的RAG问答流程，包括问题向量化、文档检索、上下文构建和答案生成四个核心环节。系统集成了多种对话模式：简单对话模式支持基础的AI聊天功能；RAG问答模式基于用户个人知识库进行检索增强回答；多模态对话模式支持图片上传和理解。创新性地实现了带引用来源的问答功能，系统通过向量检索获取相关文档片段后，提取文档元数据（文档名称、知识库来源、内容类型等），在生成回答的同时展示引用来源，用户可点击查看原始文档，有效增强了回答的可信度和可追溯性。此外，系统实现了流式输出功能，使用Server-Sent Events技术实时返回AI生成的内容，提升了用户体验。

3. 知识库管理功能

提供了知识库的创建、编辑、删除以及文档上传、下载、删除等全生命周期管理功能。系统支持多知识库架构，用户可根据不同主题或用途创建多个知识库。文档上传后，系统自动进行异步向量化处理：首先使用Apache Tika解析PDF、Word、TXT等多种格式文档，提取文本内容和元数据；然后使用TokenTextSplitter按800 token大小进行智能文本分割，设置200 token重叠以保持上下文连贯；最后使用OpenAI Embedding模型生成1536维向量，存储到PgVector向量数据库。文档删除时，系统会同步删除向量数据库中的对应向量，确保数据一致性。系统还提供文档状态监控功能，实时显示文档的向量化进度。

4. 内容管理与同步

实现了文章、游戏、学习资源、视频等多类型内容的管理功能，支持Markdown格式编辑、封面图片上传、视频文件管理、分类标签管理等。内容创建后，系统自动将内容同步到知识库，包括标题、描述、正文内容等文本信息，实现了内容发布与知识沉淀的无缝衔接，无需用户手动处理。评论系统同样支持同步到知识库，用户的评论内容经过审核后自动向量化，丰富了知识库的内容来源。系统还支持内容推荐功能，管理员可设置推荐内容在首页展示，配合Redis缓存技术，确保推荐列表的高效加载。

5. 安全与权限控制

使用Spring Security实现了基于JWT（JSON Web Token）的认证授权机制。用户登录成功后，服务器生成包含用户名、角色、过期时间等信息的JWT令牌，客户端后续请求携带该令牌进行身份验证。系统支持基于角色的访问控制（RBAC），定义了管理员和普通用户两种角色，不同角色拥有不同的操作权限。密码使用BCrypt算法进行加密存储，该算法具有自动加盐和自适应计算成本的特点，有效防止彩虹表攻击。在安全防护方面，系统使用MyBatis Plus的参数化查询防止SQL注入攻击，对用户输入进行HTML转义防止XSS攻击，配置CORS策略控制跨域访问，全方位保障系统安全。

6. 前端界面主题样式与动画设计

前端界面采用了简洁现代的白色背景设计风格，通过CSS变量（CSS Variables）统一管理界面色系，以白色为主色调，配合浅灰色边框和阴影效果，营造出干净清爽的视觉效果。界面设计注重留白和排版，使用圆角、细微阴影、渐变等视觉元素，配合毛玻璃（backdrop-filter）效果，提升了界面的层次感和质感。针对Element Plus组件库进行了深度样式定制，覆盖卡片、菜单、表格、表单等常用组件，所有组件采用统一的边框风格设计，保持视觉上的一致性和协调性。

在动画设计方面，系统集成了GSAP（GreenSock Animation Platform）动画库，实现了丰富的交互动效。主要包括：页面元素的淡入动画（fadeIn），当用户滚动页面时，内容元素以渐变和位移的方式优雅地呈现；内容切换的过渡动画，在文章列表与详情页之间切换时，通过淡入淡出效果实现平滑过渡；管理后台的头像悬停动画、按钮悬停效果等微交互设计，增强了用户的操作反馈感。这些动画效果不仅提升了视觉体验，也有效引导了用户的注意力，使界面更加生动活泼。

7. MinIO多模块文件存储设计

系统采用MinIO对象存储服务实现多模块文件存储，设计了合理的多bucket架构以满足不同类型文件的存储需求。主要包含default（默认文件）、photos（图片资源）、documents（文档资料）等多个bucket，实现了文件的分类存储和管理。系统支持bucket的自动创建和权限配置，当上传文件到不存在的bucket时，会自动创建该bucket并设置为公开读取权限，简化了运维工作。

在文件访问方面，实现了临时URL和公开URL两种访问方式。对于敏感文件，系统生成带有时效性的预签名URL（默认7天有效期），确保文件访问的安全性；对于公开资源，直接提供公开访问链接。针对图片资源，实现了原图与缩略图的分离存储，缩略图在图片上传时自动生成，有效减少带宽消耗并提升页面加载速度。所有文件的元数据（文件名、大小、类型、存储路径等）均持久化到数据库中，便于管理和追溯。

8. 系统测试与优化

进行了全面的功能测试和性能测试，验证了系统的正确性和稳定性。功能测试覆盖了AI问答、知识库管理、内容管理、评论系统等核心模块，确保各功能点符合预期设计。性能测试结果表明，系统在100并发用户下能够保持良好的响应性能，平均响应时间控制在3秒以内，满足日常使用需求。向量检索性能在10万条向量数据下仍能保持毫秒级响应，展现了良好的扩展性。

7.2 未来展望

尽管系统已经实现了核心功能，但仍有一些可以改进和扩展的方向：

1. 多模型支持

当前系统主要使用OpenAI的模型，未来可以支持更多模型提供商，如Anthropic Claude、Google Gemini、阿里云通义千问等，让用户可以根据需求选择合适的模型。

2. 混合检索优化

目前系统主要使用向量检索，可以结合关键词检索实现混合检索，在保持语义理解能力的同时提高检索的准确性和召回率。

3. 对话质量评估

引入对话质量评估机制，收集用户反馈，自动评估回答质量，持续优化检索策略和提示词模板。

4. 知识图谱集成

将知识图谱技术与RAG结合，通过实体关系推理提供更结构化的答案，特别适合处理复杂的多跳问题。

5. 私有化部署支持

优化系统架构，支持私有化部署，满足企业对数据安全和隐私保护的要求。支持本地部署的开源大模型，降低对外部API的依赖。

6. 移动端适配

进一步优化移动端界面，开发小程序或App版本，让用户可以随时随地使用智能问答功能。

参考文献

[1] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.

[2] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[J]. arXiv preprint arXiv:2312.10997, 2023.

[3] Spring Boot. Spring Boot Documentation[EB/OL]. <https://spring.io/projects/spring-boot>, 2024.

[4] Spring AI. Spring AI Reference Documentation[EB/OL]. <https://docs.spring.io/spring-ai/reference/>, 2024.

[5] Vue.js. Vue.js 3 Documentation[EB/OL]. <https://vuejs.org/guide/introduction.html>, 2024.

[6] Element Plus. Element Plus Component Library[EB/OL]. <https://element-plus.org/en-US/guide/design.html>, 2024.

[7] GreenSock. GSAP Documentation[EB/OL]. <https://greensock.com/docs/>, 2024.

[8] MinIO. MinIO Object Storage Documentation[EB/OL]. <https://min.io/docs/minio/kubernetes/upstream/>, 2024.

[9] pgvector. pgvector - Open-source vector similarity search for PostgreSQL[EB/OL]. <https://github.com/pgvector/pgvector>, 2024.

[10] OpenAI. OpenAI Embeddings API Documentation[EB/OL]. <https://platform.openai.com/docs/guides/embeddings>, 2024.

致谢

时光荏苒，四年的大学生活即将结束。在毕业论文完成之际，我要向所有给予我帮助和支持的人表示最诚挚的感谢。

首先，我要感谢我的指导老师。在论文选题、系统设计到论文撰写的整个过程中，老师都给予了悉心的指导和耐心的帮助。老师严谨的治学态度和精益求精的工作作风，让我受益匪浅。

其次，我要感谢我的父母。感谢你们多年来对我的养育之恩和无私支持，是你们的鼓励让我能够坚持完成学业，追求自己的梦想。

感谢我的同学们，在学习和生活中给予我的帮助和陪伴。我们一起讨论技术问题、分享学习心得，这段时光将成为我珍贵的回忆。

感谢开源社区的所有贡献者。本系统的开发离不开Spring Boot、Vue.js、PgVector等优秀的开源项目，正是这些开源软件降低了开发门槛，让我能够专注于业务逻辑的实现。

最后，感谢所有在百忙之中评阅本论文的老师，感谢你们的宝贵意见。

由于时间和能力有限，论文中难免存在不足之处，恳请各位老师批评指正。

全文完